

ACM – OPEN PROJECT

AutoJudge: Predicting Programming Problem Difficulty

Astha Jaiswal | 23118015 | Metallurgical and Materials Engineering

1. Introduction

Competitive programming platforms like Codeforces, CodeChef, Kattis, and LeetCode categorize programming problems into difficulty levels (Easy, Medium, Hard) and assign numerical difficulty scores. Traditionally, this classification relies on human expert judgment and user submission statistics, which is time-consuming and subjective.

AutoJudge is an automated system that predicts both the difficulty class (Easy/Medium/Hard) and numerical difficulty score (0-10 scale) using only the textual problem statement. By analyzing problem descriptions, input/output specifications, and structural patterns, our system learns to identify complexity indicators that correlate with problem difficulty.

Key Contributions:

- Purely text-based prediction without code execution or runtime data
- Dual-task learning: Classification + Regression on unified feature space
- Rich feature engineering: TF-IDF + domain-specific numeric features
- Production-ready web interface for real-time predictions

2. Problem Statement

Given a programming problem's textual description comprising:

- Problem description/story
- Input specification
- Output specification

Predict:

1. Difficulty Class: Easy / Medium / Hard (3-class classification)
2. Difficulty Score: Continuous value $\in$ (regression)

Constraints:

- Use only provided dataset (no manual labeling)
- No deep learning (classical ML only)
- Deploy as web application

3. Dataset

Dataset Overview

- Source: Kattis competitive programming problems
- Format: data.jsonl (JSON Lines)
- Size: 4,112 problems
- Class distribution: Hard(47.2%), Medium(34.2%), Easy(18.6%)
- Score range: 1.1 - 9.7 (mean=5.11, std=2.18)

Data Schema

Column	Description	Example
title	Problem title	"Uuu"
description	Problem story + task	"Unununium (Uuu) was the name..."
input_description	Input format	"The input consists of one line..."
output_description	Output format	"The output consists of N lines..."
problem_class	Target (Easy/Medium/Hard)	"hard"
problem_score	Target (float 0-10)	9.7

Train/Test Split: 80/20 stratified by problem_class

4. Methodology

4.1 Data Preprocessing

Text Combination

full_text = title + " " + description + " " + input_description + " " + output_description

Text Cleaning Pipeline

1. Lowercase conversion
2. HTML tag removal: <.*?> →
3. URL removal: https?:/\/S+|www\.\S+ →
4. Whitespace normalization: \s+ → single space

4.2 Feature Engineering

Feature Set 1: TF-IDF (30,000 features)

- N-grams: Unigrams + Bigrams (1,2)
- Max features: 30,000
- Min document frequency: 5
- Sublinear TF scaling

Feature Set 2: Handcrafted Numeric Features (22 features)

Feature Group	Examples	Rationale
Text Statistics	log(text_length), log(math_symbols)	Longer/more complex statements
Constraints	has_constraints, has_big_n (10 ⁵ +)	Algorithmic complexity indicators
Algorithm Keywords	dp_count, graph_count, segment_tree_count	Explicit difficulty signals

Final Feature Matrix

$X_{\text{final}} = \text{hstack}([X_{\text{tfidf}}(30\text{K}), X_{\text{numeric}}(22)]) \in \mathbb{R}^{(32,911)}$

4.3 Classification Model

Architecture: Multinomial Logistic Regression

- C=1.0 (optimal via GridSearchCV)
- L2 regularization
- 5-fold Stratified K-Fold CV

Baseline Comparison:

Logistic Regression CV Accuracies: [0.53647416 0.52583587 0.51367781 0.51671733 0.52054795]
Mean: 0.5226506224757463 Std: 0.008017140210626111

Random Forest CV Accuracies: [0.47416413 0.49696049 0.47264438 0.48480243 0.48401826]
Mean: 0.4825179386823223 Std: 0.008755916264603545

LinearSVC CV Accuracies: [0.51367781 0.49696049 0.49088146 0.49088146 0.48249619]
Mean: 0.4949794821260866 Std: 0.010420863211367945

4.4 Regression Model

Architecture: Gradient Boosting Regressor

- n_estimators=300
- learning_rate=0.05
- max_depth=3
- 5-fold K-Fold CV

Baseline Comparison:

Linear Regression CV RMSE: [2.44114723 75.85660012 82.59401992 115.38460446 2.14703423]
Mean RMSE: 55.684681191773166 Std: 45.59880088994406

Gradient Boosting CV RMSE: [2.01721044 2.02694623 2.02963878 1.97640871 1.99409114]
Mean RMSE: 2.0088590601763636 Std: 0.02049295061603739

5. Experimental Setup

Training Pipeline

- 1. Load data.jsonl → DataFrame(4112×8)
- 2. Drop irrelevant columns → DataFrame(4112×5)
- 3. Train/test split (80/20 stratified)
- 4. Text preprocessing → full_text_clean
- 5. TF-IDF vectorization → (3289×30000)
- 6. Numeric feature extraction → (3289×22)
- 7. Feature stacking → X_final(3289×32911)
- 8. Model training + 5-fold CV
- 9. Hyperparameter tuning (GridSearchCV/RandomizedSearchCV)
- 10. Final model training on full train set

Evaluation Metrics -

Classification: Accuracy, Precision, Recall, F1, Confusion Matrix

Regression: MAE, RMSE

6. Results and Evaluation

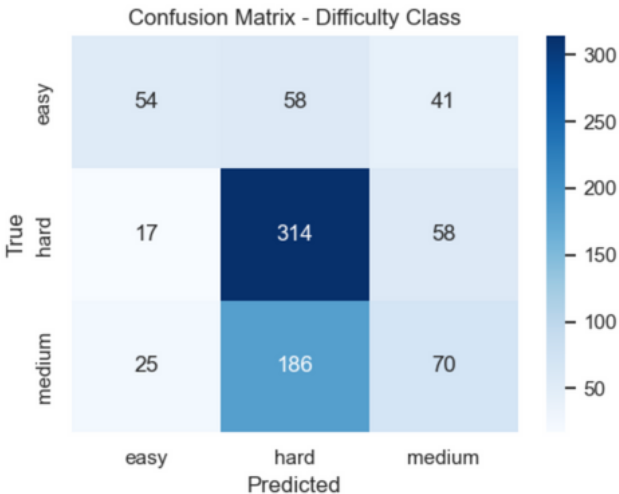
6.1 Classification Results (Test Set: n=823)

Overall Accuracy: 53.21%

Test Accuracy: 0.5321992709599028

Classification Report:	precision	recall	f1-score	support
easy	0.56	0.35	0.43	153
hard	0.56	0.81	0.66	389
medium	0.41	0.25	0.31	281
accuracy			0.53	823
macro avg	0.51	0.47	0.47	823
weighted avg	0.51	0.53	0.50	823

Confusion Matrix:
[[54 58 41]
[17 314 58]
[25 186 70]]



Key Insights:

- Hard problems most reliably detected (high recall)
- Medium problems hardest to distinguish (overlap with both Easy/Hard)
- Model captures meaningful patterns despite text-only input

6.2 Regression Results (Test Set: n=823)

Test RMSE: 2.00498587596746
Test MAE: 1.6810055660118064

6.3 Feature Importance Insights

Top TF-IDF Terms by Difficulty:

Easy: "simple", "basic", "single", "small"

Hard: "segment tree", "max flow", "dynamic programming", "10^5"

7. Web Interface

Architecture

Streamlit app → Loads 4 artifacts:

- tfidf.pkl (Vectorizer)
- logreg_classifier.pkl (Classifier)
- gb_regressor.pkl (Regressor)
- label_encoder.pkl (Decoder)

User Workflow

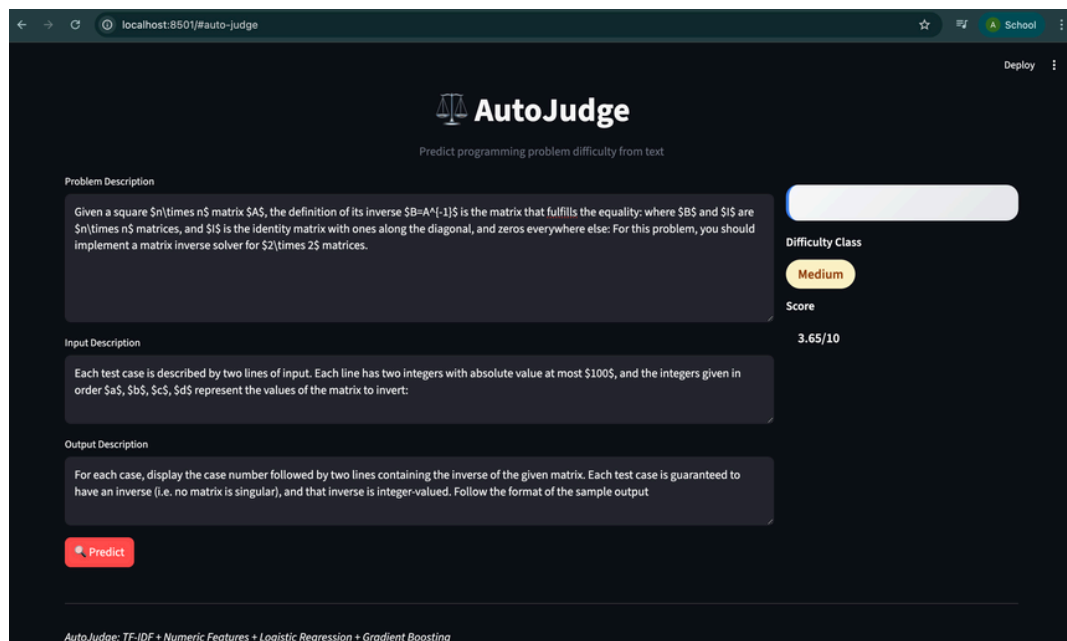
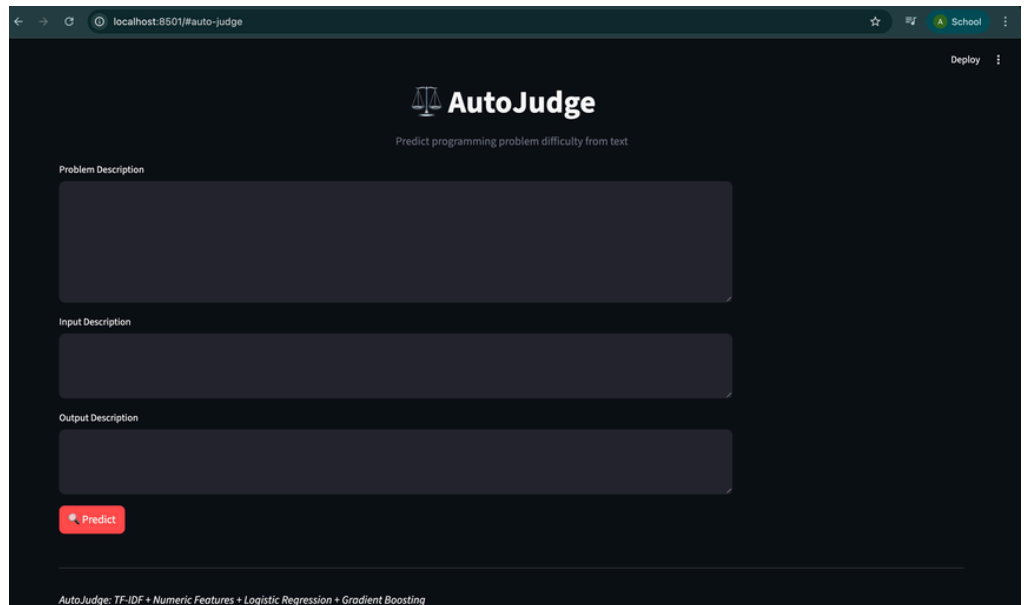
1. Paste: Problem + Input + Output descriptions
2. Click "Predict Difficulty"
3. View: Class badge + Score (0-10)

UI Screenshot

Example: Medium Problem

Input: Given a square $n \times n$ matrix A , the definition

Prediction: Medium (Score: 3.65)



8. Conclusion

Key Findings

1. Text alone predicts difficulty reasonably well (54% accuracy, RMSE=2.01)
2. Domain features significantly improve performance over TF-IDF alone
3. Hard problems easiest to identify (explicit algorithmic hints)
4. Medium problems challenging (boundary cases)

Future Work

1. BERT/RoBERTa fine-tuning for contextual understanding
2. Constraint parsing (extract N, time limits numerically)
3. Multi-lingual support for global platforms
4. Active learning with human feedback loop

Impact

AutoJudge enables automated problem triaging for:

- Contest organizers (difficulty balancing)
- Educational platforms (curriculum sequencing)
- New problem creators (self-assessment)

9. References

1. Kattis Open Dataset (data.jsonl)
2. Scikit-learn Documentation (TF-IDF, Logistic Regression, Gradient Boosting)
3. Streamlit Documentation (Web Deployment)
4. "Estimating Difficulty Levels of Programming Problems"

GitHub Repository: <https://github.com/Astha5487/AutoJudge>

Keywords: Competitive Programming, NLP, Difficulty Prediction, TF-IDF, Multi-task Learning